

# RTL with Verilog (Week#1)

## FSM (Finite State Machine)

is a mathematical model used to represent a system that can exist in a **finite number of states** and changes from one state to another based on **inputs**.

### Types of FSM (Finite State Machine)

There are **two main types of FSM**:

#### 1. Moore FSM

In a **Moore FSM**, the output depends **only on the current state**.

#### 2. Mealy FSM

In a **Mealy FSM**, the output depends on **current state and current input**.

## Three Techniques for Writing FSM in Verilog

There are **three common coding styles** used for FSM implementation.

### 1. One-Process FSM (Single Always Block)

In this style:

- State register
- Next state logic
- Output logic

are written in **one always block**.

### 2. Two-Process FSM (Most Common Method)

This separates:

1. State register
2. Next state and output logic

#### Process 1: State Register

### 3. Three-Process FSM (Professional RTL Style)

This divides FSM into:

1. State register

2. Next-state logic
3. Output logic

### **1101 Sequence Detector:**

#### **Design Code:**

```
module seq1101 (  
    input clk,  
    input rst,  
    input x,  
    output reg y  
);  
  
parameter S0 = 3'd0,  
    S1 = 3'd1,  
    S2 = 3'd2,  
    S3 = 3'd3,  
    S4 = 3'd4;  
  
reg [2:0] state, next_state;  
  
// State Register  
always @(posedge clk or posedge rst)  
begin  
    if (rst)  
        state <= S0;  
    else  
        state <= next_state;
```

end

// Next State Logic

always @(\*)

begin

case(state)

S0: next\_state = (x) ? S1 : S0;

S1: next\_state = (x) ? S2 : S0;

S2: next\_state = (x) ? S2 : S3;

S3: next\_state = (x) ? S4 : S0;

S4: next\_state = (x) ? S2 : S0;

default: next\_state = S0;

endcase

end

// Output Logic (Moore)

always @(\*)

begin

case(state)

S4: y = 1'b1;

default: y = 1'b0;

endcase

end

endmodule

### Test Bench Code:

```
`timescale 1ns/1ps
```

```
module tb_seq1101;
```

```
reg clk;
```

```
reg rst;
```

```
reg x;
```

```
wire y;
```

```
// Instantiate DUT
```

```
seq1101 uut (
```

```
    .clk(clk),
```

```
    .rst(rst),
```

```
    .x(x),
```

```
    .y(y)
```

```
);
```

```
// Clock generation (10ns period)
```

```
always #5 clk = ~clk;
```

```
// Stimulus
```

```
initial
```

```
begin
```

```
// Initialize
```

```
clk = 0;
```

```
rst = 1;
```

```
x = 0;
```

```
// Reset
```

```
#10;
```

```
rst = 0;
```

```
// Apply sequence 1101
```

```
#10 x = 1; // first 1
```

```
#10 x = 1; // second 1
```

```
#10 x = 0; // 0
```

```
#10 x = 1; // 1 --> output should become 1
```

```
// Extra bits to check overlapping
```

```
#10 x = 0;
```

```
#10 x = 1;
```

```
// Finish simulation
```

```
#20;
```

```
$finish;
```

```
end
```

```
// Monitor output
```

```
initial
```

```
begin
```

```
    $monitor("Time=%0t | x=%b | state=%d | y=%b",
```

```
        $time, x, uut.state, y);
```

```
end
```

```
// Generate waveform file
```

```
initial
```

```
begin
```

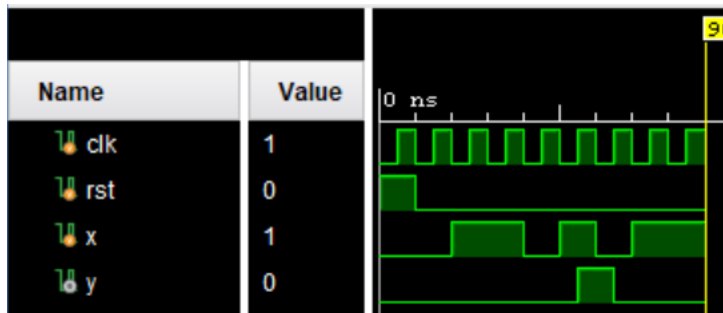
```
    $dumpfile("seq1101.vcd");
```

```
    $dumpvars(0, tb_seq1101);
```

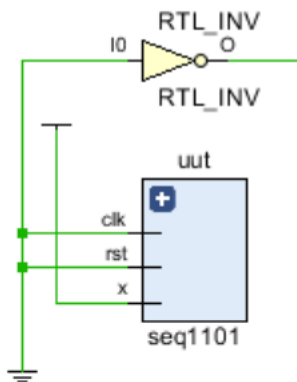
```
end
```

```
endmodule
```

**Wave form:**



**Synthesis:**



## Traffic Light FSM

**Design :**

```

module traffic_light (
    input clk,
    input rst,
    output reg [2:0] light
);

parameter RED    = 2'd0,
           GREEN  = 2'd1,
           YELLOW = 2'd2;

reg [1:0] state, next_state;

```

```

// State Register

always @(posedge clk or posedge rst)

begin

    if (rst)

        state <= RED;

    else

        state <= next_state;

end


// Next State Logic

always @(*)

begin

    case(state)

        RED:  next_state = GREEN;

        GREEN: next_state = YELLOW;

        YELLOW: next_state = RED;

        default: next_state = RED;

    endcase

end


// Output Logic

always @(*)

begin

    case(state)

```



```
    RED:   light = 3'b100; // Red ON

    GREEN: light = 3'b001; // Green ON

    YELLOW: light = 3'b010; // Yellow ON

    default: light = 3'b100;

endcase

end

endmodule
```

### **Test Bench Design :**

```
`timescale 1ns/1ps

module tb_traffic_light;

    reg clk;

    reg rst;

    wire [2:0] light;

    // Instantiate DUT

    traffic_light uut (

        .clk(clk),

        .rst(rst),

        .light(light)

    );
```

```
// Clock generation (10ns period)
```

```
always #5 clk = ~clk;
```

```
// Test stimulus
```

```
initial
```

```
begin
```

```
    // Initial values
```

```
    clk = 0;
```

```
    rst = 1;
```

```
    // Reset
```

```
    #10;
```

```
    rst = 0;
```

```
    // Wait and observe states
```

```
    #10;
```

```
    #10;
```

```
    #10;
```

```
    #10;
```

```
#10;
```

```
// Stop simulation
```

```
$finish;
```

```
end
```

```
// Display output
```

```
initial
```

```
begin
```

```
    $monitor("Time=%0t | State=%d | Light=%b",
```

```
        $time, uut.state, light);
```

```
end
```

```
initial
```

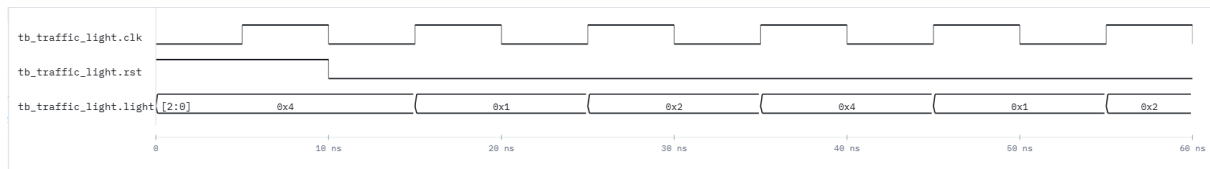
```
begin
```

```
    $dumpvars(0, tb_traffic_light);
```

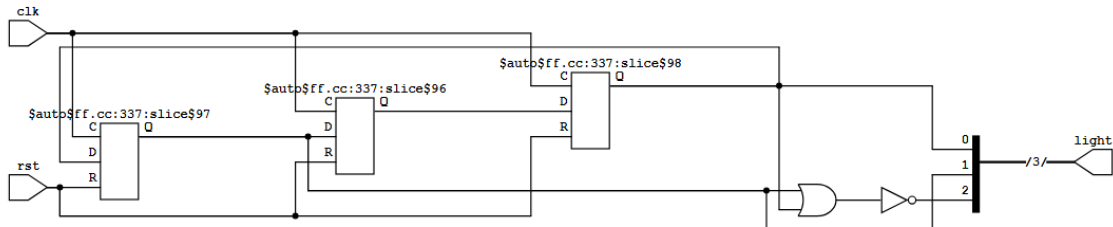
```
end
```

```
endmodule
```

**Wave Form:**



## Synthesis:



## Vending Machine:

### Design :

```
module vending_machine(
    input clk,
    input rst,
    input coin5,
    input coin10,
    output reg dispense
);
```

```
// State declaration
```

```
parameter IDLE    = 2'd0,
           FIVE    = 2'd1,
           DISPENSE = 2'd2;
```

```
reg [1:0] state, next_state;
```

```
//=====
```

```
// 1. State Register
```

```
//=====
```

```
always @(posedge clk or posedge rst)
```

```
begin
```

```
    if(rst)
```

```
        state <= IDLE;
```

```
    else
```

```
        state <= next_state;
```

```
end
```

```
//=====
```

```
// 2. Next State Logic
```

```
//=====
```

```
always @(*)
```

```
begin
```

```
case(state)
```

```
    IDLE:
```

```
    begin
```

```
        if(coin10)
```

```
            next_state = DISPENSE;
```

```
        else if(coin5)
```

```
            next_state = FIVE;
```

```
        else
```

```
            next_state = IDLE;
```

```
    end
```

```
    FIVE:
```

```
    begin
```

```
        if(coin5)
```

```
            next_state = DISPENSE;
```

```
        else
```

```
            next_state = FIVE;
```

```
    end
```

DISPENSE:

begin

next\_state = IDLE;

end

default:

next\_state = IDLE;

endcase

end

//=====

// 3. Output Logic

//=====

always @(\*)

begin

case(state)

DISPENSE:

```
dispense = 1'b1;
```

```
default:
```

```
dispense = 1'b0;
```

```
endcase
```

```
end
```

```
endmodule
```

### **Test Bench Design :**

```
`timescale 1ns/1ps
```

```
module tb_vending_machine;
```

```
reg clk;
```

```
reg rst;
```

```
reg coin5;
```

```
reg coin10;
```

```
wire dispense;
```

```
// Instantiate DUT
```



```
vending_machine uut (  
    .clk(clk),  
    .rst(rst),  
    .coin5(coin5),  
    .coin10(coin10),  
    .dispense(dispense)  
);
```

```
// Clock generation (10ns period)  
always #5 clk = ~clk;
```

```
// Test stimulus
```

```
initial
```

```
begin
```

```
    // Initial values
```

```
    clk = 0;
```

```
    rst = 1;
```

```
    coin5 = 0;
```

```
    coin10 = 0;
```

```
    // Reset
```

```
#10;
```

```
rst = 0;
```

```
// Insert 5 coin
```

```
#10 coin5 = 1;
```

```
#10 coin5 = 0;
```

```
// Insert another 5 coin
```

```
#10 coin5 = 1;
```

```
#10 coin5 = 0;
```

```
// Wait for dispense
```

```
#20;
```

```
// Insert 10 coin directly
```

```
#10 coin10 = 1;
```

```
#10 coin10 = 0;
```

```
// Finish simulation
```

```
#20;
```

```

    $finish;

end

// Monitor values

initial
begin

    $monitor("Time=%0t | coin5=%b | coin10=%b | State=%d | Dispense=%b",

        $time, coin5, coin10, uut.state, dispense);

end

initial begin

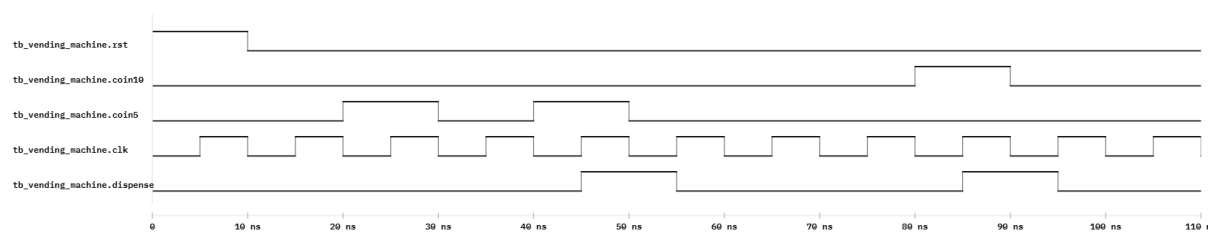
    $dumpvars(0, tb_vending_machine);

end

endmodule

```

## Wave Form:



## Synthesis:

